

GLIDE

Arbeitsvorbereitung für Codex, Build, Release und Dokumentation

Windows · macOS · Branding · Packaging · QA · Signing · Stores · Automatisierung

Zweck und Abgrenzung dieses Dokuments

Diese Revision dokumentiert den tatsächlich erreichten Source- und Strukturstand von Glide 2.5.1 sowie die weiterhin offenen Build-, Signing- und Store-Arbeiten. Codeblöcke, Buildbefehle und die Arbeitspakete C01–C08 sind Referenzmaterial beziehungsweise Vorlagen; sie sind keine eigenständigen Arbeitsanweisungen und werden nur nach einem separaten Auftrag ausgeführt.

Dokumentversion 2.5.1 · Stand 31.08.2026

Arbeitsstatus: Source 2.5.1 implementiert und getestet · Packaging/Release weiterhin Pre-Release

Inhaltsverzeichnis

1. Management Summary
2. Ausgangslage und vorhandene Unterlagen
3. Was bereits richtig vorbereitet ist
4. Ergänzungen und offene Punkte der bisherigen Vorbereitung
5. Branding- und Icon-Vorbereitung
6. Umgesetzte Gesamt- und Ordnerstruktur
7. Aufgabenverteilung: Codex vs. Benutzer
8. Empfohlene Codex-Arbeitspakete
9. Eingearbeitete AGENTS.md für Glide
10. Eingerichtetes Dokumentationssystem
11. Zentrales Produkt- und Identitätsregister
12. Apple- und Microsoft-Accountentscheidungen
13. QA: reproduzierbares Testsystem
14. Referenz-Testdaten und Migration
15. Release-Artefakte und Versionierung
16. Security und Software-Supply-Chain
17. Empfohlener Grafik- und Asset-Workflow
18. Nachgeholte Arbeitsvorbereitung und verbleibende Punkte
19. Verbindlicher Phasenplan
20. Strategische Leitprinzipien
21. Zielzustand für den späteren Release-Workflow
22. Fazit und nächste konkrete Schritte
23. Quellen und Prüfstand

Die Kapitel sind in Prozessreihenfolge angeordnet – von der Bestandsaufnahme und Arbeitsvorbereitung über Codex-Struktur, QA und Packaging bis zu Signing, Veröffentlichung und Automatisierung.

1. Management Summary

Gesamtbewertung. Glide 2.5.1 liegt als lauffähiger, getesteter Source-Stand vor. Ordnerübersicht, fester Eingang, listenübergreifendes Drag & Drop, freie Notizen, Aufgabenbeschreibungen, lokale Anhänge, Dark-Mode-Chrome und sichere Komplettbackups sind umgesetzt. Der zuvor abgebrochene 2.5.0-Stand wurde nachvollzogen und stabilisiert.

Der nächste Qualitätssprung liegt nun außerhalb der reinen Feature-Implementierung: manuelle Plattform-QA, verbindliche Produktidentitäten, reproduzierbares Packaging, Signing/Notarisierung und Clean-Machine-Tests. Die neue Struktur trennt dafür Source, Tests, Fixtures, technische Dokumentation, Grafik-Master, Store-Material und finale Release-Artefakte.

Kernaussage 2.5.1

Die gewünschte Quality-of-Life-Erweiterung ist im Source umgesetzt und automatisiert geprüft. Die Repository-Grundlage, technische Markdown-Dokumentation und getrennten Arbeitsbereiche sind angelegt. Noch nicht nachgewiesen sind reproduzierbare EXE-/DMG-Builds, Installer, Signing, Notarisierung, Clean-Machine-Tests und Store-Reife.

Die drei wichtigsten nächsten Arbeiten sind:

1. Produkt-, Publisher-, Lizenz- und Plattformidentitäten verbindlich festlegen.
2. Die manuelle Windows-/macOS-QA-Matrix auf realen Zielsystemen abschließen.
3. Reproduzierbare unsigned Builds vorbereiten; erst danach Signing, Notarisierung und Stores angehen.

2. Ausgangslage und vorhandene Unterlagen

2.1 Vorhandene Build- & Release-Anleitung

Die vorhandene Anleitung beschreibt bereits einen vollständigen Zielprozess von der Projektvorbereitung bis zur Veröffentlichung:

- Projekt und Stammdaten vorbereiten
- Branding und Icon-Master vorbereiten
- Windows mit PyInstaller bauen
- Windows-Installer mit Inno Setup erstellen
- Windows Code Signing
- macOS-App-Bundle erzeugen
- DMG erstellen
- Apple Developer ID, Signierung und Notarisierung
- Benutzerdaten, Einstellungen, Backups und Logs trennen
- Versionierung und Updatefähigkeit
- Clean-Machine- und Release-Tests
- Microsoft Store und Mac App Store
- spätere Automatisierung und KI-Unterstützung

Bewertung: Diese Basis ist solide und bleibt die zentrale technische Referenz. Die neuen Ergänzungen in diesem Dokument ersetzen sie nicht, sondern strukturieren und operationalisieren sie.

2.2 Funktionskatalog und Produktphilosophie

Der Funktionskatalog macht mehrere wichtige Produktgrenzen deutlich: Glide ist aktuell lokal ausgerichtet, benötigt kein Benutzerkonto und soll möglichst ohne externe Programmbibliotheken auskommen. Cloud-Synchronisation, serverbasierte Dienste und tiefgreifende Betriebssystemintegration sind bewusst spätere, aufwendigere Themen.

Konsequenz für Codex

Release-, Packaging- und Strukturarbeiten dürfen diese Produktphilosophie nicht stillschweigend verändern. Neue Runtime-Abhängigkeiten, Cloud-Dienste, Telemetrie, Kontensysteme oder Lizenzserver dürfen nicht „nebenbei“ eingeführt werden.

2.3 Aktuelle Arbeitsstruktur

 <code>_pycache_</code>		31.08.2026 13:06	Dateiordner	
 <code>05_Probelisten_Testdaten</code>		21.07.2026 11:48	Dateiordner	
 <code>07_Python-Versionen</code>		31.08.2026 16:02	Dateiordner	
 <code>10_Dokumentation</code>		31.08.2026 16:02	Dateiordner	
 <code>100_Archiv</code>		31.08.2026 12:08	Dateiordner	
 <code>_test_glide_v250.py</code>		31.08.2026 13:09	Python File	8 KB

Abbildung 1: Historische Ausgangslage vor der Strukturüberarbeitung (Stand 31.08.2026).

Der Screenshot zeigt den Zustand, an dem der vorherige Lauf abbrach: Versionsdateien lagen unter `07_Python-Versionen`, der Test noch im Workspace-Root und ein generierter `_pycache_` daneben. Dieser Zustand wurde nicht überschrieben, sondern nachvollziehbar in eine kanonische Repository-Struktur überführt; historische Versionen und Testdaten bleiben separat erhalten.

2.4 Umgesetzter Arbeitsstand 2.5.1

Die neue Struktur wurde nicht-destruktiv angelegt. Der getestete Source besitzt jetzt einen stabilen Pfad ohne Versionsnummer, während die leicht startbaren Versionskopien als Historie erhalten bleiben.

```
Glide_Workspace/
├── 00_Arbeitsvorbereitung/{Entscheidungen,Research,Checklisten,Notizen}/
├── 01_Repository/Glide/
│   ├── AGENTS.md · README.md · CHANGELOG.md · LICENSE.md · SECURITY.md
│   ├── .gitignore · .gitattributes · .editorconfig · VERSION
│   ├── src/glide/{app.pyw,resources}/
│   ├── tests/{unit,integration,smoke,packaging,fixtures}/
│   ├── assets/{branding,runtime,installer}/
│   ├── packaging/{pyinstaller,windows,macos}/
│   ├── scripts/{build,assets,test,release,verify}/
│   └── requirements/
├── docs/{00_INDEX,01_PRODUCT_CONSTRAINTS,02_ARCHITECTURE,05_QA_TESTPLAN,06_DATA_BACKUP_MIGRATION,10_RELEASE_CHECKLIST,decisions,exec-plans}/
├── 05_Probelisten_Testdaten/           (Legacy-Quelle)
├── 07_Python-Versionen/                 (Versionshistorie)
├── 10_Dokumentation/                   (Word/PDF)
├── 20_Grafik_Master/
├── 30_Release_Exports/<VERSION>/      (einziger Ort finaler Binärartefakte)
├── 40_Store_Material/
├── 90_Testdaten_Extern/
└── 100_Archiv/
```

Der frühere Root-Test wurde in die Repository-Tests überführt; der generierte Python-Cache wurde entfernt. Es wurde bewusst noch kein Git-Repository initialisiert und kein Packaging- oder Signingwert erfunden.

3. Was bereits richtig vorbereitet ist

Bereich	Bewertung
Windows EXE	gut vorbereitet
Windows Installer	gut vorbereitet
macOS .app	gut vorbereitet
DMG	gut vorbereitet
Windows ICO	gut vorbereitet
macOS ICNS	gut vorbereitet
Apple Icon Composer	berücksichtigt

PyInstaller OneDir	richtig gewählt
Windows Signing	berücksichtigt
Apple Developer ID	berücksichtigt
Notarisierung	berücksichtigt
Benutzerdaten außerhalb Programmordner	berücksichtigt
Versionierung	berücksichtigt
Updatefähigkeit	berücksichtigt
Clean-Machine-Tests	berücksichtigt
Microsoft Store	grundsätzlich berücksichtigt
Mac App Store	grundsätzlich berücksichtigt
KI-Arbeitsauftrag	bereits vorhanden
Release-Checkliste	bereits vorhanden

Besonders sinnvoll ist die Entscheidung für PyInstaller OneDir als Ausgangspunkt. OneDir ist leichter zu diagnostizieren, startet in der Regel schneller und bildet eine robuste Basis für Installer. Unter macOS ist ein normales App-Bundle zusätzlich die sinnvollere Grundlage für signierte bzw. sandboxed Szenarien.

4. Ergänzungen und offene Punkte der bisherigen Vorbereitung

4.1 Microsoft Store: Silent Installation und unveränderliche Installer

Für eine klassische EXE/MSI-Einreichung sollte die vorhandene Anleitung um folgende Store-Anforderungen ergänzt werden:

- Silent Installation ohne Benutzerinteraktion muss funktionieren.
- Der Installer darf kein Webstub/Downloader sein, der wesentliche Bestandteile erst während des Setups nachlädt.
- Download-URLs sollten versioniert und dauerhaft eindeutig sein.
- Eine bereits eingereichte Datei hinter einer Download-URL darf nicht stillschweigend ersetzt werden.
- Installer und relevante PE-Binaries müssen digital signiert sein.
- Returncodes und Silent-Uninstall sollten für einen professionellen Win32-Release definiert und getestet werden.

```
Glide-Setup-<VERSION>.exe /VERYSILENT /SUPPRESSMSGBOXES /NORESTART
```

4.2 Inno Setup: dauerhaft stabile AppId

Die Inno-Setup-AppId sollte einmalig erzeugt und anschließend über alle Glide-Versionen stabil gehalten werden. Sie identifiziert die Installation für Upgrade- und Deinstallationslogik. Eine versehentliche Änderung kann zu parallelen Installationen oder doppelten Deinstallationseinträgen führen.

```
AppId={{EINMALIG-GENERIERTE-STABILE-GUID}}
```

Wichtig ist die Trennung der Identitäten:

- Inno Setup AppId – Installationsidentität
- Windows AppUserModelID – Taskleisten-/Shell-Zuordnung
- macOS Bundle Identifier – Apple-Bundle-Identität
- Produktname – sichtbarer Marken-/Anwendungsname
- Versionsnummer – Release-Version

4.3 Windows Code Signing: SHA-256 und RFC-3161 Timestamping

Für öffentliche Windows-Releases sollte die Signaturstrategie nicht nur „signieren“, sondern auch den Zeitstempel festlegen:

```
SHA-256 Signatur
+
RFC-3161 Timestamp
```

Das Timestamping stellt sicher, dass eine korrekt signierte Veröffentlichung auch nach Ablauf des verwendeten Zertifikats weiterhin als zum Signierzeitpunkt gültig bewertet werden kann. Automatisierung sollte erst erfolgen, wenn der unsigned Build reproduzierbar und stabil funktioniert.

4.4 Apple Notarisierung: finalen Workflow präzisieren

Für den produktiven macOS-Release sollte der Ablauf klar in technische Prüfschritte zerlegt werden:

```
Glide.app bauen
↓
alle eingebetteten Binaries prüfen
↓
Glide.app mit Developer ID signieren
↓
Signatur verifizieren
↓
DMG erzeugen
↓
DMG signieren
↓
finale DMG mit notarytool an Apple senden
↓
Notarisierung prüfen
↓
Ticket staplen
↓
Gatekeeper-Test
```

Für aktuelle Apple-Workflows ist notarytool der relevante CLI-Weg. Hardened Runtime und korrekte Signierung der eingebetteten Bestandteile sind zentrale Voraussetzungen.

4.5 Marken-/Namensprüfung für „Glide“

Vor größerem Aufwand für Store-Grafiken, öffentliches Branding und Veröffentlichung sollte eine Namens-/Markenprüfung erfolgen. Es existiert bereits ein international aktives Softwareangebot unter dem Namen Glide/GlideOS. Das bedeutet nicht automatisch, dass die Bezeichnung für die eigene ToDo-App unzulässig ist, erhöht aber die Relevanz einer professionellen Prüfung.

- DPMA-Recherche
- EUIPO-Recherche
- relevante Waren-/Dienstleistungsklassen für Software
- Store-Namensverfügbarkeit
- Domain-/Webauftritt
- mögliche Verwechslungsgefahr

Hinweis

Die Markenprüfung in diesem Dokument ist eine Projekt- und Risikomanagement-Empfehlung, keine Rechtsberatung.

5. Branding- und Icon-Vorbereitung

Die bisherige Spezifikation mit einem hochwertigen 1024 × 1024 px Master, Vektorquelle und reproduzierbaren Exporten ist richtig. Technische Formate sollten aus einem kontrollierten Master erzeugt werden statt als voneinander unabhängige Masterdateien gepflegt zu werden.

5.1 Empfohlenes Masterpaket

```
Glide_Icon_Master.ai
Glide_Icon_Master.svg
Glide_Icon_Master.psd
Glide_Icon_Master_1024.png

Apple_Layers/
├─ 01_Background.svg
├─ 02_BaseShape.svg
├─ 03_MainSymbol.svg
```

```
└─ 04_Foreground.svg
└─ 05_Highlight.svg
```

Für Apple Icon Composer sollten getrennte, sinnvoll benannte Ebenen auf einem 1024 × 1024-Canvas vorbereitet werden. Vektor ist bevorzugt; Text sollte in Konturen umgewandelt werden. Systemmasken und bestimmte systemseitige Material-/Schatteneffekte sollten nicht unnötig in das Masterartwork eingebrannt werden.

5.2 Small-Size-Prüfung

Prüfgröße	Typische Verwendung
1024 px	Master / Store / große Vorschau
256 px	große Windows-Shell-Darstellung
128 px	hochauflösende Zwischengröße
64 px	Systemdarstellung
48 px	Explorer / Taskleiste bei Skalierung
32 px	Taskleiste / Start / Titelleiste
24 px	Taskleiste / Suche
16 px	Kontextmenü / kleine Shell-Ansicht

Für sehr kleine Größen kann eine optisch vereinfachte Variante sinnvoll sein, sofern feine Linien oder Details sonst verschwinden. Unter Windows werden zusätzlich Zwischenwerte durch Display Scaling relevant; der Multi-Resolution-ICO bleibt für klassische Win32-Apps dennoch der zentrale technische Container.

5.3 Trennung von Win32-ICO und späteren Store-/MSIX-Assets

Für eine klassische Windows-EXE reicht ein korrekt aufgebautes Multi-Resolution-ICO. Ein späteres MSIX-Paket benötigt dagegen ein separates Package-/Store-Assetset. Diese Assets sollten nicht ungeordnet mit den klassischen ICO-Ressourcen vermischt werden.

6. Umgesetzte Gesamt- und Ordnerstruktur

Die empfohlene Zweiteilung ist für 2.5.1 angelegt: ein persönlicher Workspace für Arbeitsmaterialien und ein technisch kontrollierter Repository-Bereich für Source, Tests, Packaging-Vorbereitung und code-nahe Dokumentation.

```
Glide_Workspace/
├─ 00_Arbeitsvorbereitung/
│   ├── Entscheidungen/
│   ├── Research/
│   ├── Checklisten/
│   └─ Notizen/
├─ 01_Repository/
│   └─ Glide/
│       ├── AGENTS.md
│       ├── README.md
│       ├── CHANGELOG.md
│       ├── LICENSE.md
│       ├── SECURITY.md
│       ├── .gitignore
│       ├── .gitattributes
│       ├── .editorconfig
│       ├── VERSION
│       ├── src/
│       │   └─ glide/
│       │       ├── __init__.py
│       │       ├── __main__.py
│       │       ├── app.py
│       │       ├── version.py
│       │       ├── paths.py
│       │       └─ resources/
│       ├── tests/
│       │   ├── unit/
│       │   ├── integration/
│       │   └─ smoke/
```

```

├── packaging/
├── fixtures/
├── assets/
│   ├── branding/
│   │   ├── master/
│   │   ├── windows/
│   │   └── macos/
│   └── apple-icon-composer/
├── runtime/
├── installer/
├── packaging/
│   ├── pyinstaller/Glide.spec
│   ├── windows/
│   │   ├── Glide.iss
│   │   ├── version_info.txt
│   │   └── signing/
│   └── macos/
│       ├── entitlements.plist
│       ├── dmg/
│       └── signing/
├── scripts/
│   ├── build/
│   ├── assets/
│   ├── test/
│   ├── release/
│   └── verify/
├── requirements/
│   ├── runtime.txt
│   ├── dev.txt
│   ├── build-windows.txt
│   └── build-macos.txt
├── docs/
│   ├── 00_INDEX.md
│   ├── 01_PRODUCT_CONSTRAINTS.md
│   ├── 02_ARCHITECTURE.md
│   ├── 03_BUILD_RELEASE.md
│   ├── 04_BRANDING_ASSETS.md
│   ├── 05_QA_TESTPLAN.md
│   ├── 06_DATA_BACKUP_MIGRATION.md
│   ├── 07_SECURITY_SIGNING.md
│   ├── 08_STORE_PREPARATION.md
│   ├── 09_SUPPORT_PRIVACY_LICENSE.md
│   ├── 10_RELEASE_CHECKLIST.md
│   ├── decisions/
│   └── exec-plans/
├── release/
│   ├── staging/
│   ├── manifests/
│   ├── notes/
│   └── .github/workflows/
├── 20_Grafik_Master/
│   ├── Illustrator/
│   ├── Photoshop/
│   ├── Branding/
│   └── Arbeitsdateien/
├── 30_Release_Exports/
│   ├── Windows/
│   ├── macOS/
│   └── Hashes/
├── 40_Store_Material/
│   ├── Microsoft/
│   └── Apple/
├── 90_Testdaten_Extern/
└── 100_Archiv/

```

6.1 Zweck der Zweiteilung

Glide_Workspace ist der persönliche Designer-/Projektarbeitsplatz. Dort dürfen AI-/PSD-Arbeitsdateien, Research, Screenshots, Entwürfe und Archive liegen.

01_Repository/Glide ist die technisch kontrollierte Softwarebasis. Nur dort arbeitet später Codex produktiv. Das reduziert das Risiko, dass alte Python-Kopien, Archive oder Teststände als aktueller Source interpretiert werden.

6.2 Umgang mit der aktuellen Struktur

Aktuell	Empfehlung
__pycache__	erledigt: generierten Cache entfernt; im Repository per .gitignore ausgeschlossen
05_Probelisten_Testdaten	aufgearbeitet: Legacy-v2-Fixture unter tests/fixtures; TXT-Beispiele unter 90_Testdaten_Extern
07_Python-Versionen	als Historie beibehalten; kanonischer Source ist src/glide/app.pyw ohne Versionsnummer
10_Dokumentation	umgesetzt: technische Markdown-Doku im Repository; Word-Dokumentation bleibt außerhalb
100_Archiv	bleibt außerhalb des produktiven Repository-Bereichs
_test_glide_v250.py	erledigt: kanonischer Integrationstest unter tests/integration/test_glide.py; alte Testkopie archiviert

Wichtig für den nächsten Schritt

Der funktionsfähige Monolith wurde gesichert und geprüft; die neue Struktur enthält eine bytgleiche kanonische Source-Kopie. Eine inhaltliche Modulzerlegung ist weiterhin ein separates Refactoring und darf erst mit zusätzlichen Regressionstests erfolgen.

7. Aufgabenverteilung: Was Codex übernehmen kann und was beim Benutzer bleibt

Aufgabe	Codex	Benutzer
Repository analysieren	vollständig	Kontrolle
Import-/Dependency-Analyse	vollständig	–
Source-Struktur planen	vollständig	Freigabe
Dateien verschieben/refactoren	vollständig	Review
.gitignore / .gitattributes / .editorconfig	vollständig	–
AGENTS.md erstellen	vollständig	Regeln freigeben
Dokumentationsstruktur	vollständig	Inhalte freigeben
Tests erstellen	weitgehend	fachliche Prüfung
Smoke Tests	weitgehend	reale Endkontrolle
Datenpfade Windows/macOS	vollständig	–
Migrationen	weitgehend	Backup-/Datenfreigabe
PyInstaller Spec	vollständig	–
Windows Buildskripte	vollständig	ausführen/testen
Inno Setup Script	vollständig	Installer testen
Windows Version Resources	vollständig	Publisherdaten liefern
ICO erzeugen	technisch	Artwork liefern
ICNS erzeugen	technisch	Artwork liefern
Icon-Validator	vollständig	optische Kontrolle
DMG-Skript	vollständig	optische Kontrolle
DMG-Hintergrund technisch einbauen	vollständig	gestalten
Windows Signierskript	vollständig	Zertifikat/Account
Apple Signierskript	vollständig	Developer Account
Notarisierungsskript	vollständig	Credentials/Secrets

CI/CD	vollständig	Secrets hinterlegen
SHA-256 Manifest	vollständig	–
Release Notes	vollständig	Freigabe
Store-Texte	sehr gut	Freigabe
Store-Screenshots automatisieren	teilweise	finale Design
Datenschutzerklärung entwerfen	Entwurf	rechtliche Prüfung
Lizenz/EULA entwerfen	Entwurf	rechtliche Entscheidung
Markenprüfung recherchieren	Unterstützung	finale rechtliche Prüfung
Apple Developer Account	nein	Benutzer/Organisation
Microsoft Partner Account	nein	Benutzer/Organisation
Zertifikate anlegen/kaufen	Anleitung	Benutzer/Organisation
Finales Publishing	vorbereiten	Benutzer/Organisation

Rollenverständnis: Codex sollte für Glide primär als Software-/Release-Engineer eingesetzt werden. Das hochwertige Artwork und die Produktentscheidungen bleiben beim Designer bzw. Projektverantwortlichen; Codex übernimmt die technische Ableitung, Validierung, Builds, Tests und Automatisierung.

```

Master-Asset
↓
Validierung
↓
Windows ICO / macOS ICNS / Store Assets
↓
Build
↓
Installer / DMG
↓
Signing
↓
Tests
↓
Release Manifest

```

8. Empfohlene Codex-Arbeitspakete

Die folgenden Blöcke C01–C08 sind wiederverwendbare Ticketvorlagen und Statusreferenzen. Sie sind Inhalt dieses Dokuments, keine automatisch auszuführenden Anweisungen. C01 wurde für den vorhandenen Workspace lesend nachvollzogen; die Grundlage aus C02 ist teilweise umgesetzt. C03 ist teilweise umgesetzt, C04–C08 sind nicht begonnen und benötigen jeweils einen separaten Auftrag.

C01 – Repository Audit (für 2.5.1 abgeschlossen)

Analysiere das vollständige Glide-Projekt ausschließlich lesend.

Nimm keine Änderungen am Sourcecode oder an Dateien vor.

Erstelle:

1. vollständige Projektstruktur
2. Liste aller produktiv verwendeten Python-Dateien
3. Import- und Dependency-Analyse
4. Liste aller Runtime-Assets
5. Liste aller hartcodierten Dateipfade
6. Speicherorte der Benutzerdaten
7. mögliche Packaging-Probleme
8. mögliche macOS-/Windows-Inkompatibilitäten
9. Liste alter, redundanter oder möglicherweise archivierter Dateien
10. Risikoanalyse einer späteren Repository-Umstrukturierung

Unterscheide ausdrücklich zwischen produktivem Sourcecode, Testcode, Testdaten, Dokumentation, generierten Dateien und Archivmaterial.

Nimm keinerlei Refactoring vor.

C02 – Repository Foundation (Grundlage umgesetzt)

Erstelle auf Basis des freigegebenen Audit-Berichts eine professionelle Repository-Struktur für Glide.

Ziele:

- konventionelle Python-Projektstruktur
- keine funktionalen Änderungen
- keine Änderung des Datenformats
- keine Änderung der Oberfläche
- keine neuen Runtime-Abhängigkeiten
- alle bisherigen Funktionen müssen weiter funktionieren

Erstelle zusätzlich:

```
.gitignore
.gitattributes
.editorconfig
AGENTS.md
README.md
VERSION
docs/00_INDEX.md
```

Dokumentiere jede Verschiebung und führe anschließend bestehende Tests bzw. Smoke Tests aus.

C03 – Release-Härtung (teilweise umgesetzt)

Prüfe Glide auf Release-Tauglichkeit.

Implementiere bzw. dokumentiere:

- plattformgerechte Datenpfade
- robuste Resource-Pfade für Source- und PyInstaller-Modus
- zentrale Versionsverwaltung
- Datenformat-/Schema-Version
- Backup vor Migrationen
- Logging
- Fehlerbehandlung
- Mehrfachstartverhalten
- Upgrade-Kompatibilität

Keine neuen Produktfunktionen hinzufügen.

C04 – Asset Pipeline (nicht begonnen)

Erstelle für Glide eine reproduzierbare Asset-Pipeline.

Input:

assets/branding/master/

Output Windows:

- Glide.ico

Output macOS:

- Glide.icns

Store-Assets:

- getrennte Exportordner

Prüfe Abmessungen, Transparenz, Farbraum soweit automatisierbar, Dateinamen und benötigte Auflösungen.

Verändere das Artwork nicht gestalterisch.

C05 – Windows Build (nicht begonnen)

Erstelle einen reproduzierbaren Windows-Releaseprozess für Glide.

Ziel:

Source → PyInstaller OneDir → Glide.exe → Inno Setup → Glide-Setup-x.y.z.exe

Berücksichtige:

- Glide.ico
- GUI-Fenstericon
- AppUserModelID
- Windows Version Resources
- stabile Inno-AppId
- Upgrade vorhandener Versionen
- Deinstallation

- Nutzerdatenerhalt
- Silent Install
- Silent Uninstall
- Logging
- Release-Dateinamen

C06 – macOS Build (nicht begonnen)

Erstelle einen reproduzierbaren macOS-Releaseprozess.

Source → PyInstaller → Glide.app → Codesign-ready App Bundle → Glide.dmg

Berücksichtige:

- Glide.icns
- Bundle Identifier
- CFBundleVersion
- CFBundleShortVersionString
- Mindest-macOS-Version
- arm64/x86_64 Entscheidung
- Hardened Runtime
- Entitlements
- Developer-ID-Signing
- notarytool
- Stapling
- Gatekeeper-Verifikation

C07 – QA (Source-/Integrationstest vorhanden; Release-QA offen)

Erstelle und automatisiere den Glide Release-Testplan.

Testbereiche:

- Unit
- Integration
- Smoke
- Packaging
- Installation
- Update
- Datenmigration
- Backup
- Restore
- Deinstallation
- Reinstallation
- Start nach Neustart
- fehlende Assets
- Windows DPI Scaling
- macOS Gatekeeper
- Signaturprüfung

Erzeuge einen maschinenlesbaren Testreport.

C08 – Release Automation (nicht begonnen)

Automatisiere nach erfolgreicher Stabilisierung den Glide Release-Prozess.

Erzeuge:

- Builds Skripte Windows
- Builds Skripte macOS
- Assetgenerierung
- Testpipeline
- Versionsprüfung
- Release-Manifest
- SHA-256-Hashes
- Release Notes
- Changelog-Unterstützung

Signing Secrets dürfen niemals im Repository gespeichert werden.

9. Eingearbeitete AGENTS.md für Glide

Eine kurze, verbindliche AGENTS.md ist jetzt unter 01_Repository/Glide/AGENTS.md vorhanden. Der folgende Block bleibt ein historisches Beispiel; maßgeblich ist die lokale Datei zusammen mit den verlinkten Markdown-Dokumenten.

```
# Glide – Codex Instructions
```

```

## Product
Glide ist eine lokale Desktop-ToDo-Anwendung für Windows und macOS.

Aktuelle Grundprinzipien:
- lokale Datenspeicherung
- kein Benutzerkonto
- keine Cloudpflicht
- keine Netzwerkabhängigkeit für Kernfunktionen
- möglichst keine zusätzlichen Runtime-Abhängigkeiten
- Windows und macOS sind gleichwertige Zielplattformen

## Primary rule
Bestehende Funktionen und bestehende Nutzerdaten dürfen durch Build-, Packaging-, Dokumentations- oder
Strukturarbeiten nicht beschädigt werden.

## Source of truth
Produkt: docs/01_PRODUCT_CONSTRAINTS.md
Architektur: docs/02_ARCHITECTURE.md
Build und Release: docs/03_BUILD_RELEASE.md
Branding: docs/04_BRANDING_ASSETS.md
QA: docs/05_QA_TESTPLAN.md
Daten und Migration: docs/06_DATA_BACKUP_MIGRATION.md
Signing: docs/07_SECURITY_SIGNING.md
Stores: docs/08_STORE_PREPARATION.md
Release: docs/10_RELEASE_CHECKLIST.md

## Development rules
Vor strukturellen Änderungen:
1. aktuellen Zustand analysieren
2. Risiken dokumentieren
3. bestehende Tests ausführen
4. erst danach Dateien verändern

## Dependencies
Neue Runtime-Abhängigkeiten dürfen nicht ohne ausdrückliche Freigabe eingeführt werden.
Build- und Development-Abhängigkeiten dürfen separat verwaltet werden.

## User data
Benutzerdaten dürfen niemals im Installationsverzeichnis gespeichert werden.
Windows: %APPDATA%\Glide\
macOS: ~/Library/Application Support/Glide/

## Compatibility
Daten aus einer veröffentlichten älteren Glide-Version dürfen nicht ohne Migration unlesbar werden.
Vor Datenmigrationen muss ein Backup erstellt werden.

## Packaging
Windows-Builds werden auf Windows erstellt.
macOS-Builds werden auf macOS erstellt.
PyInstaller wird primär im OneDir-Modus verwendet.

## Security
Keine Passwörter, API-Keys, Zertifikate, PFX-Dateien, Provisioning-Profile oder Signing-Secrets
committen.

## Scope
Release-Arbeiten dürfen keine neuen Produktfunktionen implementieren, sofern diese nicht ausdrücklich
Bestandteil des Arbeitsauftrags sind.

## Completion
Nach jeder Änderung dokumentieren:
- geänderte Dateien
- Zweck der Änderung
- ausgeführte Tests
- Testergebnis
- offene Risiken

```

10. Eingerichtetes Dokumentationssystem

Das Markdown-Grundgerüst ist unter 01_Repository/Glide/docs/ eingerichtet. Diese Word-Datei bleibt die ausführliche menschlich lesbare Arbeitsgrundlage; die Markdown-Dateien bilden die knappen code-nahen Quellen. Noch nicht angelegte Spezialdokumente folgen erst zusammen mit Packaging, Signing und Store-Arbeit.

Datei	Zweck
-------	-------

00_INDEX.md	Navigation und Dokumentationsindex
01_PRODUCT_CONSTRAINTS.md	Produktgrenzen und bewusste Nicht-Ziele
02_ARCHITECTURE.md	Source, Datenmodell, Module, Datenfluss
03_BUILD_RELEASE.md	technische Build-/Release-Anleitung
04_BRANDING_ASSETS.md	Logo-, Icon-, Store- und Exportregeln
05_QA_TESTPLAN.md	vollständiger Testplan
06_DATA_BACKUP_MIGRATION.md	Speicherorte, Schema, Backup, Restore
07_SECURITY_SIGNING.md	Zertifikate, Secrets, Hashes, Signing
08_STORE_PREPARATION.md	Apple-/Microsoft-Anforderungen
09_SUPPORT_PRIVACY_LICENSE.md	Datenschutz, Lizenz, Support
10_RELEASE_CHECKLIST.md	Go/No-Go-Checkliste
CHANGELOG.md	Änderungshistorie
SECURITY.md	Sicherheitsprinzipien
AGENTS.md	Codex-Navigation und dauerhaft geltende Regeln

10.1 PRODUCT_CONSTRAINTS.md als Schutz vor KI-Fehlentscheidungen

Diese Datei verhindert, dass Codex technisch plausible, aber strategisch unerwünschte Entscheidungen automatisch einführt – etwa SQLAlchemy, platformdirs, Cloudservices, Telemetrie oder ein Kontensystem.

```
# Product Constraints

## Current architecture
Glide ist Local First.
Kernfunktionen müssen ohne Internet funktionieren.
Keine Anmeldung ist erforderlich.
Benutzerdaten verlassen den Rechner nicht.

## Runtime dependencies
Neue externe Python-Runtime-Bibliotheken nur nach ausdrücklicher Entscheidung.
Buildtools wie PyInstaller zählen nicht als Runtime-Abhängigkeit.

## Future features
Folgende Funktionen dürfen bei Release-Härtung nicht vorweggenommen werden:
- Cloud Sync
- Lizenzserver
- Benutzerkonten
- Push-Infrastruktur
- Telemetrie
- externe Analytics

## Compatibility
Datenkompatibilität hat Priorität vor internem Refactoring.
```

11. Zentrales Produkt- und Identitätsregister

Vor dem ersten öffentlichen Release sollten die technischen Stammdaten nicht an mehreren Stellen verstreut gepflegt werden. Für die Arbeitsvorbereitung empfiehlt sich eine Datei wie:

```
00_Arbeitsvorbereitung/Entscheidungen/PRODUCT_IDENTITY.md

Datei:
01_Repository/Glide/docs/decisions/PRODUCT_IDENTITY.md

Produktname:
Glide

Langname:
Glide – Aufgaben und Listen

Aktuelle App-Version:
2.5.1

Datenformat-Version:
4
```

```

Release-Kanal:
internal / pre-release

Publisher, Copyright, Support-E-Mail, Website, Datenschutz-URL und Lizenzmodell:
offen – nicht durch Annahmen ersetzen

Windows:
Ziel Windows 10/11; Architektur, AppUserModelID und Inno-Setup-AppId final festlegen

macOS:
Bundle Identifier, Mindestversion und Architektur final festlegen

```

12. Apple- und Microsoft-Accountentscheidungen früh treffen

12.1 Apple Developer Program

Vor öffentlichem Vertrieb sollte entschieden werden, ob als Einzelperson oder Organisation veröffentlicht wird. Bei einer Organisation sind Unternehmensdaten und je nach Apple-Prozess unter anderem eine D-U-N-S-Nummer sowie ein geeigneter geschäftlicher Web-/E-Mail-Auftritt relevant. Der veröffentlichte Verkäufer-/Entwicklername hängt von der gewählten Kontostruktur ab.

Diese Entscheidung sollte vor finaler Store-Gestaltung und vor dem Festlegen dauerhaft verwendeter Identitäten getroffen werden.

12.2 Microsoft Partner Center

Auch Microsoft unterscheidet Individual- und Company-Konten. Die Kontostruktur sollte frühzeitig festgelegt werden, da Publisher-Identität, Verifikation und spätere Store-Verwaltung davon abhängen. Account- und Identitätsentscheidungen sind nicht an Codex delegierbar.

13. QA: vom Checklistenpunkt zum reproduzierbaren Testsystem

Die QA-Grundlage ist jetzt operationalisiert: Der Integrationstest startet Glide mit einem isolierten temporären Datenordner und prüft Kernfunktionen, Migration, Backup/Restore sowie mehrere Fehlerfälle. Unit-, Packaging- und reale Clean-Machine-Tests bleiben als nächste Ebenen offen.

```

tests/
├── unit/                               (vorbereitet)
├── integration/test_glide.py           (grün)
├── smoke/                             (vorbereitet)
├── packaging/                         (vorbereitet)
└── fixtures/{current_v4, legacy_v2}/

```

Nachweis am 31.08.2026: „Glide v2.5.1 Kern-, Backup- und Migrationstests: OK“. Die nachfolgende Matrix ist eine Soll-Matrix; „Ja“ bedeutet erforderlich, nicht bereits für einen Installer oder DMG nachgewiesen.

Ebene	Ziel
Unit	Einzelne Logikbausteine isoliert prüfen.
Integration	Zusammenspiel von Daten, Speicherung, Laden und Modulen prüfen.
Smoke	App starten und wichtigste Kernfunktionen in kurzer Sequenz prüfen.
Packaging	Tatsächlich gebaute EXE, .app, Installer bzw. DMG testen.

13.1 Release-Testmatrix

Test	Windows	macOS
Source-Start	Ja	Ja
PyInstaller Build	Ja	Ja
Clean Install	Ja	Ja

Erster Start	Ja	Ja
Daten anlegen	Ja	Ja
Neustart	Ja	Ja
Daten vorhanden	Ja	Ja
Update Alt → Neu	Ja	Ja
Backup	Ja	Ja
Restore	Ja	Ja
Uninstall	Ja	–
Nutzerdaten erhalten	Ja	Ja
Reinstall	Ja	Ja
Icon	Ja	Ja
DPI/Scaling	Ja	Retina/Skalierung
Signatur	Ja	Ja
Offline	Ja	Ja
Clean Machine ohne Python	Ja	Ja

14. Referenz-Testdaten und Migration

Die Referenzdaten sind nun getrennt: ein kleines aktuelles Format-4-Fixture sowie ein vorhandener größerer Format-2-Bestand für Migration. Manuelle TXT-Importbeispiele liegen außerhalb der automatisierten Tests.

```
01_Repository/Glide/tests/fixtures/
├── current_v4/reference_v4.json
└── legacy_v2/probelisten_5_listen_v2.json

90_Testdaten_Extern/Legacy_Probelisten/
└── manuelle_TXT-Importbeispiele
```

Das aktuelle Fixture enthält Eingang, Ordner, Seitennotiz, Beschreibung, Fälligkeit, Wichtigkeit und Unterpunkt. Der größere Altbestand ist ausdrücklich als Legacy-Migrationsfixture gekennzeichnet. Reale personenbezogene oder vertrauliche Unternehmensdaten gehören nicht in automatisierte Fixtures.

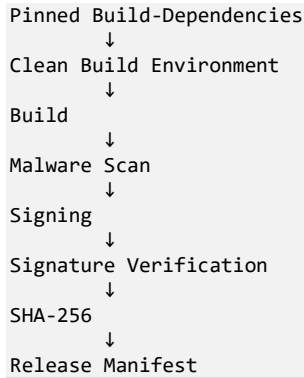
15. Release-Artefakte und Versionierung

Finale Veröffentlichungen werden ausschließlich außerhalb des Repository-Bereichs unter 30_Release_Exports/<VERSION>/ abgelegt. Für 2.5.1 ist die Struktur vorbereitet, aber noch kein EXE-, Installer-, App- oder DMG-Artefakt als freigegeben vorhanden. Das folgende Schema ist daher eine Vorlage.

```
30_Release_Exports/
├── <VERSION>/
│   ├── Windows/Glide-Setup-<VERSION>-<ARCH>.exe
│   ├── macOS/Glide-<VERSION>-<ARCH>.dmg
│   ├── RELEASE_NOTES.md
│   ├── release-manifest.json
│   └── Hashes/SHA256SUMS.txt
└── {
    "product": "Glide",
    "version": "<VERSION>",
    "channel": "<CHANNEL>",
    "build_date": "<YYYY-MM-DD>",
    "windows": {"architecture": "<ARCH>"},
    "macos": {"architecture": "<ARCH>"}
  }
```

Ein final freigegebenes Release-Artefakt sollte danach nicht mehr unter identischem Dateinamen ausgetauscht werden. Korrekturen erhalten eine neue Version bzw. Buildnummer.

16. Security und Software-Supply-Chain



Besonders wichtig ist die Trennung von Build-Konfiguration und geheimen Signierdaten. Folgende Dateien bzw. Geheimnisse dürfen niemals ins Repository:

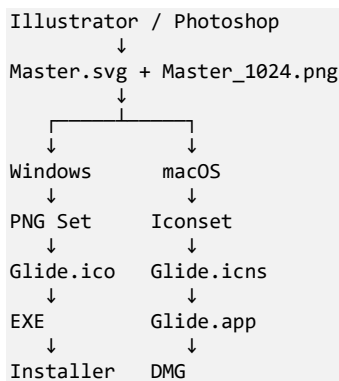
- *.pfx
- *.p12
- *.key
- *.pem
- Apple API Keys
- Notary Credentials
- Signing Tokens
- Passwörter und Recovery-Codes

Repository-Regel

Signing-Secrets gehören in Betriebssystem-Keychains, Secret Stores oder CI-Secrets. Das Repository enthält ausschließlich Skripte und Konfiguration, niemals private Schlüssel.

17. Empfohlener Grafik- und Asset-Workflow

Die grafische Arbeit bleibt in Illustrator/Photoshop. Die technische Größen- und Formatvielfalt wird anschließend reproduzierbar erzeugt.



Das Artwork wird nicht durch Codex „verbessert“ oder gestalterisch verändert. Codex validiert Größen, Dateiformate und Exportvollständigkeit und automatisiert die Ableitung technischer Assets.

18. Nachgeholte Arbeitsvorbereitung und verbleibende Punkte

18.1 Priorität 1 – funktionierenden Stand einfrieren

Abgeschlossen: Der lauffähige Source-Stand 2.5.1 ist unter 07_Python-Versionen erhalten und bytegleich in den kanonischen Repository-Pfad übernommen.

Glide_PreRelease_2.5.1_2026-08-31

Der Monolith wurde nicht in Module zerlegt; die Strukturänderung beschränkt sich auf eine getestete kanonische Kopie und begleitende Projektdateien.

18.2 Priorität 2 – Produktidentität festlegen

- Publisher
- Copyright
- Supportkontakt
- Website
- Bundle Identifier
- Windows AppUserModelID
- Inno Setup Appld
- macOS Mindestversion
- macOS Architektur
- Windows Architektur
- Lizenzmodell
- Release-Kanäle
- Datenformat-Version

18.3 Priorität 3 – Branding-Master vorbereiten

```
Glide_Icon_Master.ai
Glide_Icon_Master.svg
Glide_Icon_Master.psd
Glide_Icon_Master_1024.png

Apple-Layer:
01_Background.svg
02_Base.svg
03_Symbol.svg
04_Foreground.svg
```

Anschließend optische Tests in 1024, 256, 128, 64, 48, 32, 24 und 16 px durchführen.

18.4 Priorität 4 – Referenz-Testdaten definieren

Mindestens typische, erledigte, terminierte, priorisierte, verschachtelte und ungewöhnliche Einträge vorbereiten. Sonderzeichen, Umlaute, sehr lange Texte und leere Zustände bewusst berücksichtigen.

18.5 Priorität 5 – Dokumentationsgerüst anlegen

```
AGENTS.md                               vorhanden
docs/
├── 00_INDEX.md                         vorhanden
├── 01_PRODUCT_CONSTRAINTS.md           vorhanden
├── 02_ARCHITECTURE.md                  vorhanden
├── 05_QA_TESTPLAN.md                   vorhanden
├── 06_DATA_BACKUP_MIGRATION.md          vorhanden
├── 10_RELEASE_CHECKLIST.md             vorhanden
├── decisions/PRODUCT_IDENTITY.md       vorhanden
├── exec-plans/2.5.1-stabilisierung.md   vorhanden
└── Build/Branding/Signing/Store-Details folgen mit den jeweiligen Arbeitspaketen
```

18.6 Priorität 6 – Store-Grafiken noch nicht final produzieren

Store-Screenshots sollten erst erstellt werden, wenn UI und Release-Build nahezu eingefroren sind. So werden unnötige Mehrfachproduktionen vermieden. Für einen späteren Mac-App-Store-Release sind unter anderem 16:10-Screenshotformate relevant; die jeweils aktuellen Spezifikationen sollten unmittelbar vor der Einreichung erneut geprüft werden.

19. Verbindlicher Phasenplan

Phase	Verantwortung	Ergebnis
Phase 0 – Arbeitsvorbereitung	Benutzer	teilweise abgeschlossen: Projekt gesichert,

		Testdaten und Dokumentationsgerüst vorhanden; Identität und Branding offen
Phase 1 – Audit	Codex	für den vorhandenen Stand abgeschlossen: Source, Abhängigkeiten, Pfade, Daten, Risiken und Abbruchpunkt geprüft
Phase 2 – Repository-Härtung	Codex	teilweise abgeschlossen: Struktur, AGENTS, Doku und Integrationstest vorhanden; Git-Initialisierung und Modulzerlegung offen
Phase 3 – Asset-Pipeline	Benutzer + Codex	nicht begonnen: freigegebenes Artwork fehlt
Phase 4 – Windows Release Candidate	Codex + Benutzer	nicht begonnen: Packaging-/Installer-Konfiguration fehlt
Phase 5 – macOS Release Candidate	Codex + Benutzer	nicht begonnen: macOS-Buildumgebung und Identitäten fehlen
Phase 6 – QA	Codex + Benutzer	teilweise: Source-/Integrationstest grün; Packaging-, Clean-Machine- und Plattform-QA offen
Phase 7 – Signing	Benutzer + Codex	Windows Authenticode, Apple Developer ID, Notarisierung; Secrets außerhalb Repository
Phase 8 – Direct Release	Benutzer	signierte EXE/DMG direkt veröffentlichen; reale Nutzer-/Clean-Machine-Erfahrung sammeln
Phase 9 – Stores	Benutzer + Codex	Microsoft Store zuerst; Mac App Store anschließend mit Sandbox-/Entitlement-Prüfung
Phase 10 – Automatisierung	Codex	Build → Test → Sign → Package → Verify → Hash → Release weitgehend skript-/CI-gestützt

20. Strategische Leitprinzipien

Automatisierungsprinzip

Funktioniert manuell → ist dokumentiert → ist reproduzierbar → ist getestet → wird automatisiert.

Die umgekehrte Reihenfolge – zuerst komplexe Automatisierung zu bauen und anschließend zu hoffen, dass der technische Prozess stimmt – ist insbesondere bei PyInstaller, Installer, Signierung, Notarisierung und Datenmigration unnötig riskant.

Zweites Leitprinzip: Keine strukturelle Änderung ohne reproduzierbaren Ausgangstest. Vor dem Verschieben oder Refactoring produktiver Dateien soll Codex zunächst die tatsächlichen Imports, Pfade, Assets und Datenzugriffe des aktuellen Projekts analysieren.

Drittes Leitprinzip: Feature-Entwicklung und Release-Härtung getrennt behandeln. Neue Produktfunktionen dürfen nicht stillschweigend in Packaging-/Build-Tickets hineinrutschen.

21. Zielzustand für den späteren Release-Workflow

```

Benutzer:
Version und Design freigeben
↓
Codex:
Code analysieren
Tests ausführen
Version prüfen/setzen
Assets validieren
↓
Windows-Workflow:
EXE → Installer → Tests
↓
macOS-Workflow:
.app → DMG → Tests
↓
Release-Workflow:

```

```

Hashes → Manifest → Release Notes
      ↓
Benutzer:
Finale Freigabe
      ↓
Signing / Veröffentlichung

```

Dieser Zielzustand reduziert manuelle Wiederholungsarbeit, ohne die finale fachliche, gestalterische, rechtliche oder accountgebundene Freigabe aus der Hand zu geben.

22. Fazit und nächste konkrete Schritte

Der zuvor unterbrochene Entwicklungsstand ist als Glide 2.5.1 stabilisiert, getestet und dokumentiert. Die gewünschte Bedienungsverbesserung ist umgesetzt; die verbleibenden Risiken liegen nicht mehr im Kern-Source, sondern vor allem in manueller Plattform-QA, Produktidentität, Branding, Packaging, Signing, Notarisierung und Store-Prozessen.

Empfohlene Reihenfolge nach Abschluss von 2.5.1:

1. Den neuen 2.5.1-Stand mit realen Arbeitsdaten in einer Kopie manuell prüfen.
2. Windows 10/11: NumLock, CapsLock, Dark Mode, Menüs, Dialoge und Drag & Drop vollständig testen.
3. Produktidentität, Lizenz, Supportkontakt und Plattform-IDs verbindlich festlegen.
4. Branding-Master und Apple-Layer liefern und validieren.
5. PyInstaller-OneDir-Build reproduzierbar und zunächst unsigned erstellen.
6. Windows-Installer und macOS-App/DMG auf Clean Machines testen.
7. Erst danach Signing, Notarisierung, Hashes und Release-Manifest umsetzen.
8. Store-Material erst auf Basis eines nahezu eingefrorenen Release Candidates erstellen.
9. Eine Modulzerlegung des Monolithen als eigenes, testgestütztes Refactoring planen.

Nächster sinnvoller Arbeitsauftrag

Nach fachlicher Prüfung von 2.5.1 sollte C03 als klar begrenztes Packaging-/Release-Härtungs-Ticket folgen: zuerst Produktidentitäten festlegen, dann einen reproduzierbaren unsigned PyInstaller-OneDir-Build und passende Smoke Tests erstellen. C04–C08 bleiben separate Aufträge.

23. Quellen und Prüfstand

Interne, in dieser Revision tatsächlich geprüfte Projektgrundlagen:

- Ursprüngliche Benutzeranforderung, UI-Screenshots und gewünschte Quality-of-Life-Funktionen, Stand 31.08.2026.
- Glide-Source v2.4.1, angefangener Zwischenstand v2.5.0 und stabilisierter Source v2.5.1.
- Lokaler Workspace, neue Struktur unter 01_Repository/Glide sowie grüner Integrationstest für v2.5.1.
- Technische Markdown-Dokumentation unter 01_Repository/Glide/docs/ und Versions-/Änderungsdateien im Repository-Root.
- Hinweis: Im Workspace lagen keine eigenständige Build-/Release-Anleitung und kein separater Funktionskatalog als lokale Datei vor; Aussagen dazu aus der Vorversion sind daher Planungsbezug, kein in dieser Revision reproduzierter Buildnachweis.

Externe offizielle bzw. primäre Referenzen, die für die Ergänzungen herangezogen wurden:

- [OpenAI – Introducing the Codex app](#)
- [OpenAI – Introducing Codex](#)
- [OpenAI – Harness engineering](#)
- [PyInstaller – Operating Mode](#)
- [Microsoft Learn – App package requirements for MSI/EXE apps](#)
- [Microsoft Learn – App icon construction](#)
- [Inno Setup Help – Appld](#)
- [Microsoft Learn – Artifact Signing certificate management](#)
- [Microsoft Learn – Artifact Signing integrations](#)

- [Apple Developer – Notarizing macOS software before distribution](#)
- [Apple Developer – Icon Composer](#)
- [Apple Developer – Program enrollment](#)
- [Apple Developer – Program membership/enrollment](#)
- [Apple Developer – App Store Connect screenshot specifications](#)
- [Apple Developer – macOS distribution](#)
- [Microsoft Learn – Open a developer account](#)
- [Glide – vorhandenes Softwareangebot zur Namens-/Markenrisikoprüfung](#)

Prüfstand: 31.08.2026. Source, Struktur und automatisierter Integrationstest entsprechen App-Version 2.5.1 und Datenformat 4. Store-, Signierungs- und Plattformanforderungen können sich ändern und müssen unmittelbar vor einer Einreichung oder Zertifikatsbeschaffung anhand offizieller Quellen erneut geprüft werden.